

12. 다형성과 설계

#0.강의/1.자바로드맵/2.자바-기본

- /좋은 객체 지향 프로그래밍이란?
- /다형성 - 역할과 구현 예제1
- /다형성 - 역할과 구현 예제2
- /다형성 - 역할과 구현 예제3
- /OCP(Open-Closed Principle) 원칙
- /문제와 풀이
- /정리

좋은 객체 지향 프로그래밍이란?

객체 지향 특징

- 추상화
- 캡슐화
- 상속
- 다형성

객체 지향 프로그래밍

- 객체 지향 프로그래밍은 컴퓨터 프로그램을 명령어의 목록으로 보는 시각에서 벗어나 여러 개의 독립된 단위, 즉 "**객체**"들의 **모임**으로 파악하고자 하는 것이다. 각각의 **객체**는 **메시지**를 주고받고, 데이터를 처리할 수 있다. (**협력**)
- 객체 지향 프로그래밍은 프로그램을 **유연**하고 **변경**이 용이하게 만들기 때문에 대규모 소프트웨어 개발에 많이 사용된다.

유연하고, 변경이 용이?

- 레고 블록 조립하듯이
- 키보드, 마우스 갈아 끼우듯이
- 컴퓨터 부품 갈아 끼우듯이
- 컴포넌트를 쉽고 유연하게 변경하면서 개발할 수 있는 방법



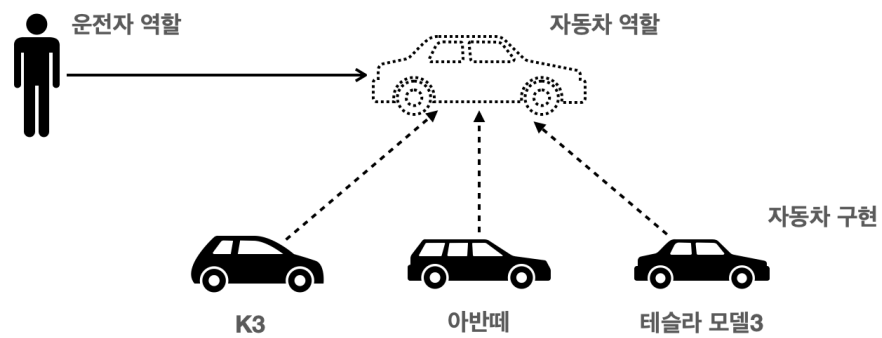
다형성

Polymorphism

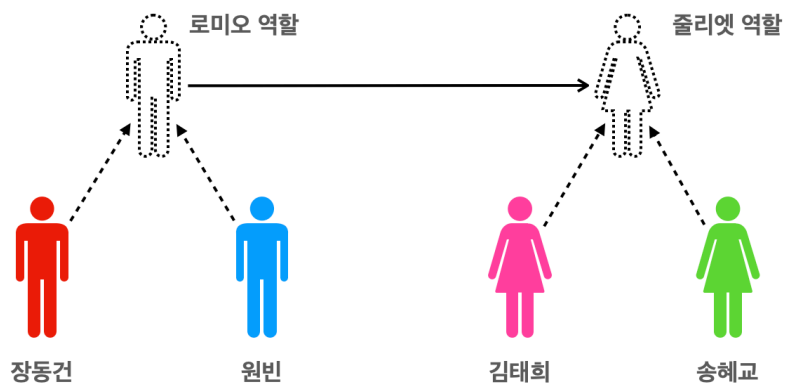
다형성의 실세계 비유

- 실세계와 객체 지향을 1:1로 매칭X
- 그래도 실세계의 비유로 이해하기에는 좋음
- 역할과 구현으로 세상을 구분

운전자 - 자동차



공연 무대 로미오와 줄리엣 공연



다형성의 실세계 비유

예시

- 운전자 - 자동차
- 공연 무대
- 키보드, 마우스, 세상의 표준 인터페이스들
- 정렬 알고리즘
- 할인 정책 로직

역할과 구현을 분리

- **역할과 구현으로 구분하면 세상이 단순해지고, 유연해지며 변경도 편리해진다.**
- 장점
 - 클라이언트는 대상의 역할(인터페이스)만 알면 된다.
 - 클라이언트는 구현 대상의 내부 구조를 몰라도 된다.
 - 클라이언트는 구현 대상의 내부 구조가 변경되어도 영향을 받지 않는다.
 - 클라이언트는 구현 대상 자체를 변경해도 영향을 받지 않는다.

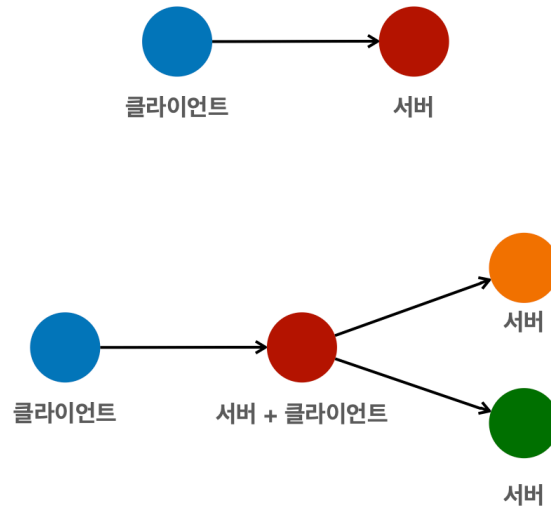
역할과 구현을 분리

자바 언어

- 자바 언어의 다형성을 활용
 - 역할 = 인터페이스
 - 구현 = 인터페이스를 구현한 클래스, 구현 객체
- 객체를 설계할 때 **역할**과 **구현**을 명확히 분리
- 객체 설계시 역할(인터페이스)을 먼저 부여하고, 그 역할을 수행하는 구현 객체 만들기

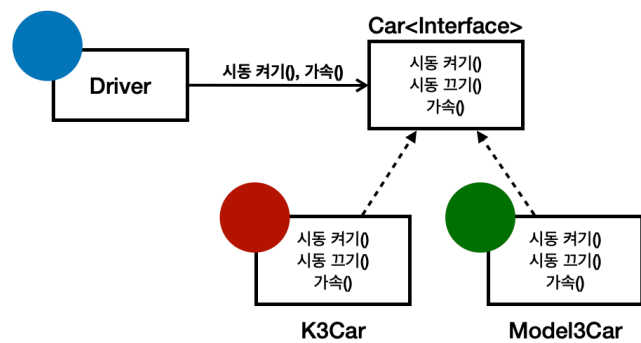
객체의 협력이라는 관계부터 생각

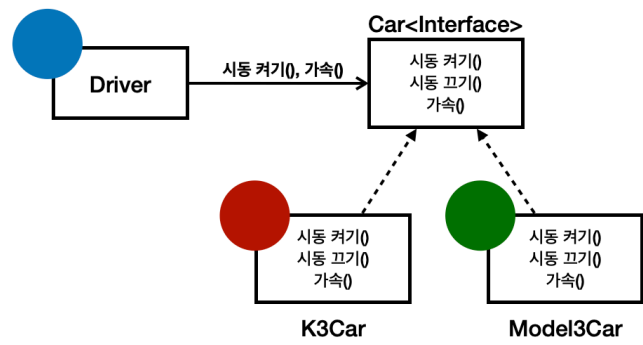
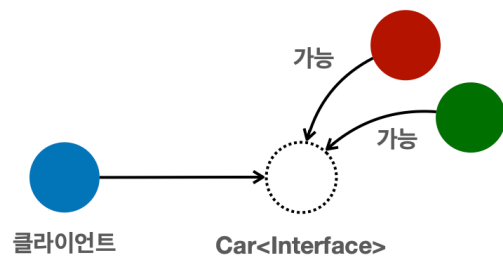
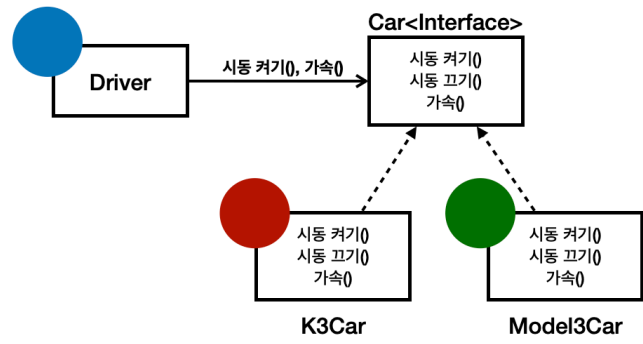
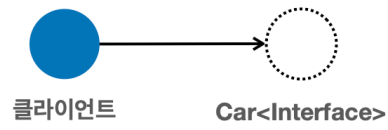
- 혼자 있는 객체는 없다.
- 클라이언트: 요청, 서버: 응답
- 수 많은 객체 클라이언트와 객체 서버는 서로 협력 관계를 가진다.

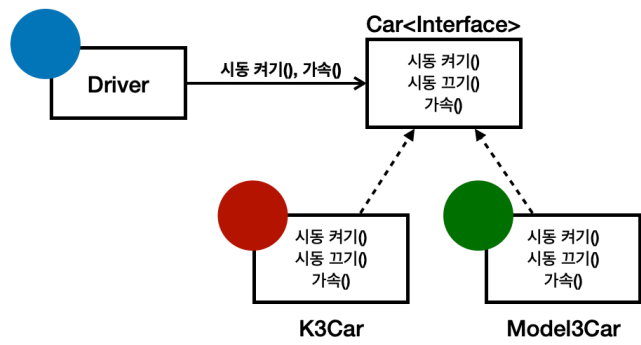
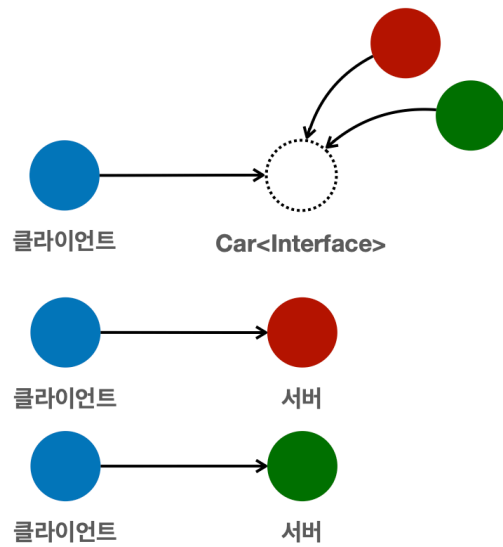
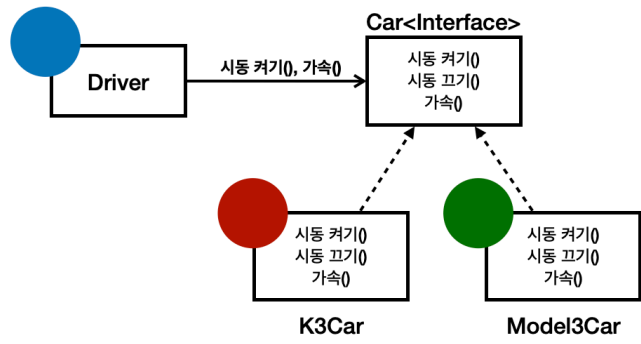
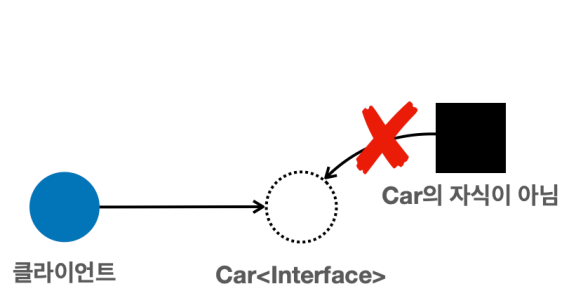


자바 언어의 다형성

- 오버라이딩을 떠올려보자
- 오버라이딩은 자바 기본 문법
- 오버라이딩 된 메서드가 실행
- 다형성으로 인터페이스를 구현한 객체를 실행 시점에 유연하게 변경할 수 있다.
- 물론 클래스 상속 관계도 다형성, 오버라이딩 적용가능







다형성의 본질

- 인터페이스를 구현한 객체 인스턴스를 실행 시점에 유연하게 변경할 수 있다.
- 다형성의 본질을 이해하려면 협력이라는 객체사이의 관계에서 시작해야함
- 클라이언트를 변경하지 않고, 서버의 구현 기능을 유연하게 변경할 수 있다.

역할과 구현을 분리 정리

- 실세계의 역할과 구현이라는 편리한 컨셉을 다형성을 통해 객체 세상으로 가져올 수 있음
- 유연하고, 변경이 용이
- 확장 가능한 설계
- 클라이언트에 영향을 주지 않는 변경 가능
- 인터페이스를 안정적으로 잘 설계하는 것이 중요

역할과 구현을 분리 한계

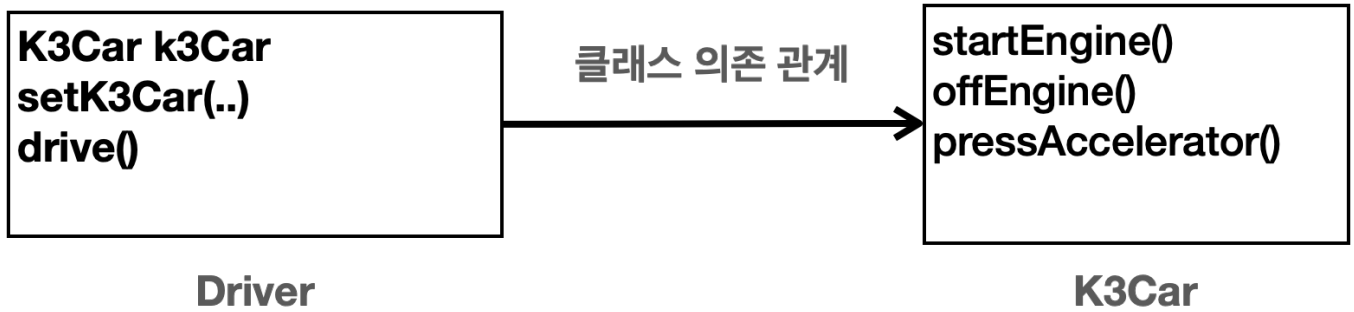
- 역할(인터페이스) 자체가 변하면, 클라이언트, 서버 모두에 큰 변경이 발생한다.
- 자동차를 비행기로 변경해야 한다면?
- 대본 자체가 변경된다면?
- USB 인터페이스가 변경된다면?
- 인터페이스를 안정적으로 잘 설계하는 것이 중요

정리

- 다형성이 가장 중요하다!
- 디자인 패턴 대부분은 다형성을 활용하는 것이다.
- 스프링의 핵심인 제어의 역전(IOC), 의존관계 주입(DI)도 결국 다형성을 활용하는 것이다.
- 스프링을 사용하면 마치 레고 블록 조립하듯이! 공연 무대의 배우를 선택하듯이! 구현을 편리하게 변경할 수 있다.

다형성 - 역할과 구현 예제1

앞서 설명한 내용을 더 깊이있게 이해하기 위해, 간단한 운전자와 자동차의 관계를 개발해보자. 먼저 다형성을 사용하지 않고, 역할과 구현을 분리하지 않고 단순하게 개발해보자.



`Driver`는 `K3Car`를 운전하는 프로그램이다.

```
package poly.car0;

public class K3Car {
    public void startEngine() {
        System.out.println("K3Car.startEngine");
    }

    public void offEngine() {
        System.out.println("K3Car.offEngine");
    }

    public void pressAccelerator() {
        System.out.println("K3Car.pressAccelerator");
    }
}
```

```
package poly.car0;

public class Driver {

    private K3Car k3Car;

    public void setK3Car(K3Car k3Car) {
        this.k3Car = k3Car;
    }

    public void drive() {
        System.out.println("자동차를 운전합니다.");
    }
}
```

```

        k3Car.startEngine();
        k3Car.pressAccelerator();
        k3Car.offEngine();
    }
}

```

```

package poly.car0;

public class CarMain0 {

    public static void main(String[] args) {
        Driver driver = new Driver();
        K3Car k3Car = new K3Car();
        driver.setK3Car(k3Car);
        driver.drive();
    }
}

```

- Driver 와 K3Car 를 먼저 생성한다. 그리고 driver.setK3Car(..) 를 통해 driver 에게 k3Car 의 참조를 넘겨준다.
- driver.drive() 를 호출한다.

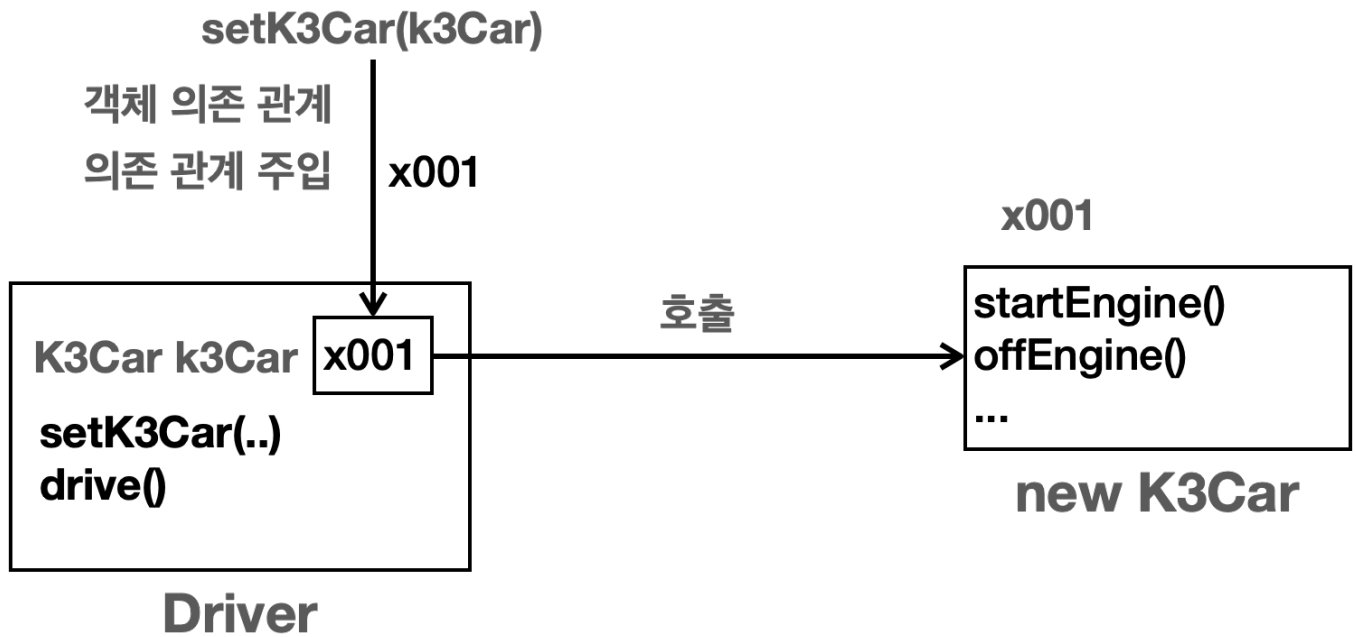
실행 결과

```

자동차를 운전합니다.
K3Car.startEngine
K3Car.pressAccelerator
K3Car.offEngine

```

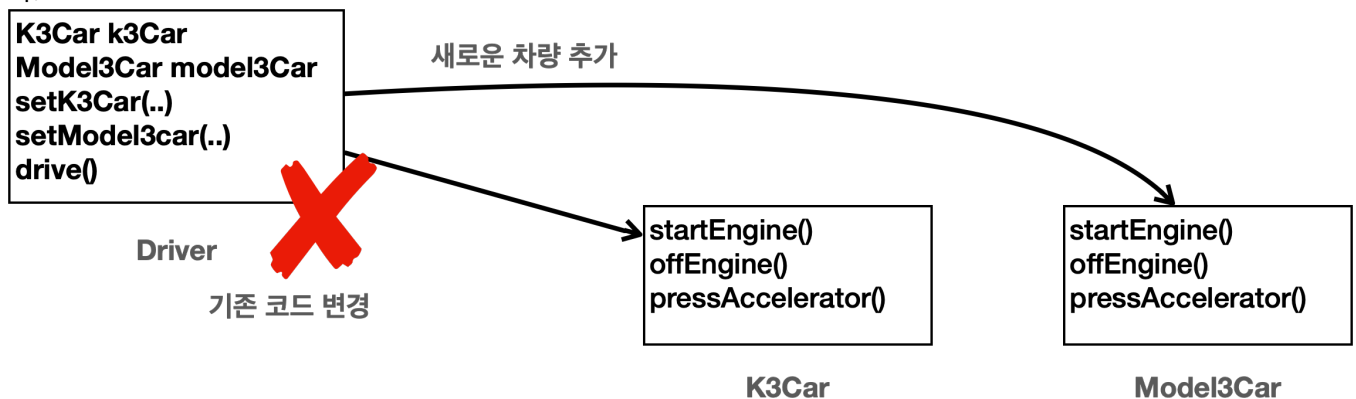
메모리 그림



다형성 - 역할과 구현 예제2

새로운 Model3 차량을 추가해야 하는 요구사항이 들어왔다. 이 요구사항을 맞추려면 기존에 Driver 코드를 많이 변경해야 한다.

드라이버는 K3Car 도 운전할 수 있고, Model3Car 도 운전할 줄 있어야 한다. 참고로 둘을 동시에 운전하는 것은 아니다.



```
package poly.car0;

public class Model3Car {
    public void startEngine() {
        System.out.println("Model3Car.startEngine");
    }
}
```

```

    public void offEngine() {
        System.out.println("Model3Car.offEngine");
    }

    public void pressAccelerator() {
        System.out.println("Model3Car.pressAccelerator");
    }

}

```

Driver - 코드 변경

```

package poly.car0;

public class Driver {

    private K3Car k3Car;
    private Model3Car model3Car; //추가

    public void setK3Car(K3Car k3Car) {
        this.k3Car = k3Car;
    }

    //추가
    public void setModel3Car(Model3Car model3Car) {
        this.model3Car = model3Car;
    }

    //변경
    public void drive() {
        System.out.println("자동차를 운전합니다.");
        if (k3Car != null) {
            k3Car.startEngine();
            k3Car.pressAccelerator();
            k3Car.offEngine();
        } else if (model3Car != null) {
            model3Car.startEngine();
            model3Car.pressAccelerator();
            model3Car.offEngine();
        }
    }

}

```

드라이버는 K3Car, Model3Car 를 모두 운전할 줄 알아야 한다. 다음과 같은 코드 변경이 발생한다.

- Model3Car 용 필드 추가
- setModel3Car(..) 메서드 추가
- drive() 메서드에서 가지고 있는 차량에 따른 분기

CarMain0 - 코드 변경

```
package poly.car0;

public class CarMain0 {

    public static void main(String[] args) {
        Driver driver = new Driver();
        K3Car k3Car = new K3Car();
        driver.setK3Car(k3Car);
        driver.drive();

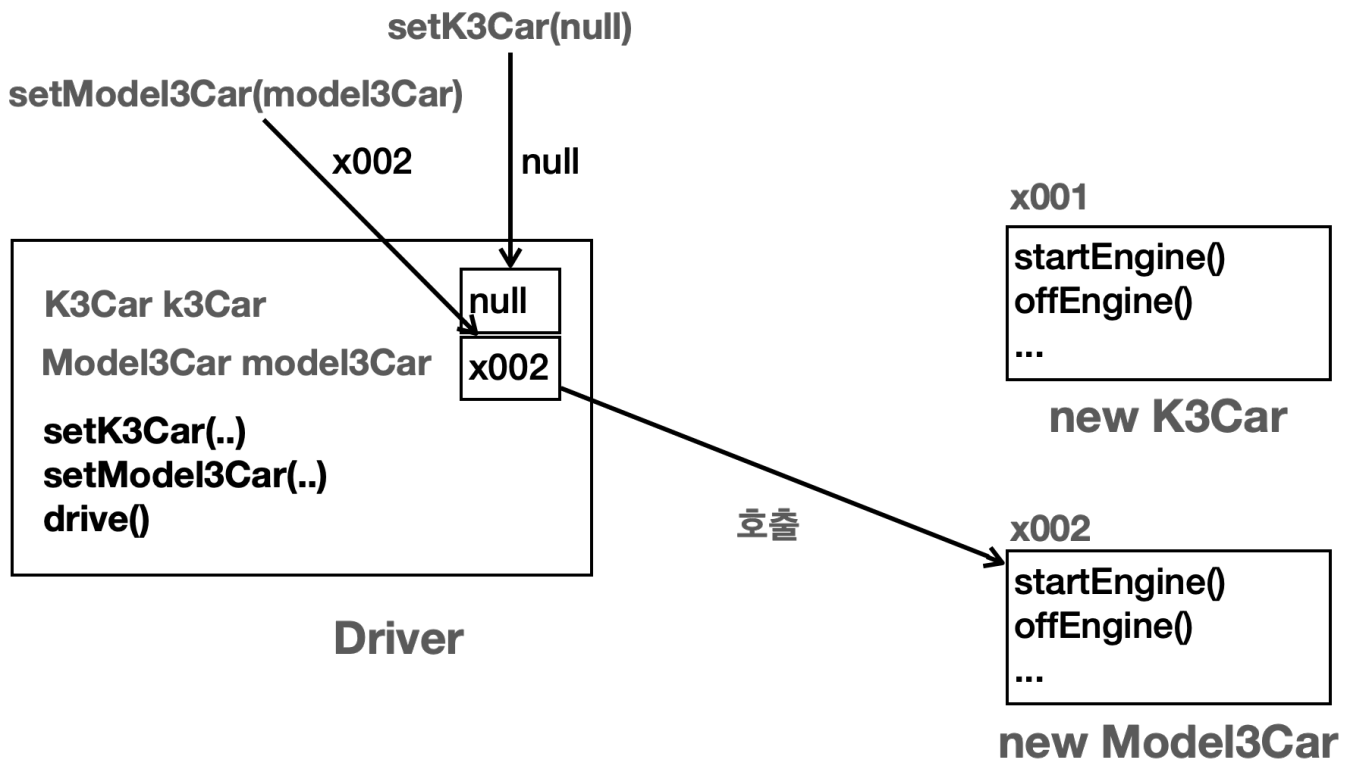
        //추가
        Model3Car model3Car = new Model3Car();
        driver.setK3Car(null);
        driver.setModel3Car(model3Car);
        driver.drive();
    }
}
```

- K3를 운전하던 운전자가 Model3로 차량을 변경해서 운전하는 코드이다.
- driver.setK3Car(null) 을 통해서 기존 K3Car 의 참조를 제거한다.
- driver.setModel3Car(model3Car) 를 통해서 새로운 model3Car 의 참조를 추가한다.
- driver.driver() 를 호출한다.

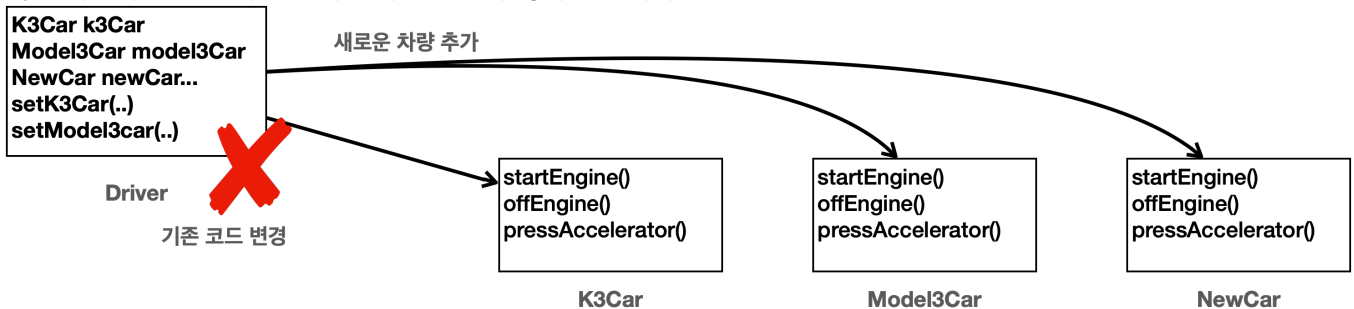
실행 결과

```
자동차를 운전합니다.
K3Car.startEngine
K3Car.pressAccelerator
K3Car.offEngine
자동차를 운전합니다.
Model3Car.startEngine
Model3Car.pressAccelerator
Model3Car.offEngine
```


메모리 그림



여기서 새로운 차량을 추가한다면 또 다시 **Driver** 코드를 많이 변경해야 한다. 만약 운전할 수 있는 차량의 종류가 계속 늘어난다면 점점 더 변경해야 하는 코드가 많아질 것이다.

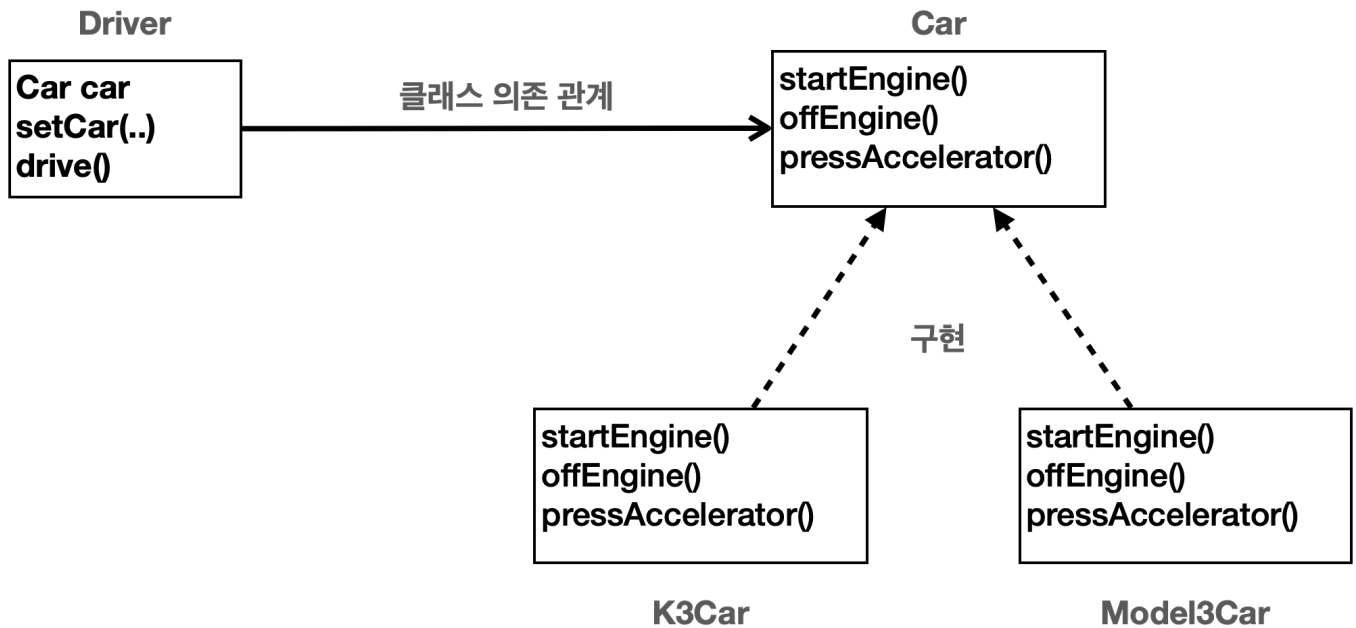


다형성 - 역할과 구현 예제3

다형성을 활용하면 역할과 구현을 분리해서, 클라이언트 코드의 변경 없이 구현 객체를 변경할 수 있다.

다음 관계에서 **Driver**가 클라이언트이다.

예제를 통해서 자세히 알아보자.



앞서 설명한 자동차 예제를 코드로 구현해보자.

- **Driver**: 운전자는 자동차(**Car**)의 역할에만 의존한다. 구현인 **K3**, **Model3** 자동차에 의존하지 않는다.
 - **Driver** 클래스는 **Car car** 멤버 변수를 가진다. 따라서 **Car** 인터페이스를 참조한다.
 - 인터페이스를 구현한 **K3Car**, **Model3Car**에 의존하지 않고, **Car** 인터페이스에만 의존한다.
 - 여기서 설명하는 의존은 클래스 의존 관계를 뜻한다. 클래스 상에서 어떤 클래스를 알고 있는가를 뜻한다. **Driver** 클래스 코드를 보면 **Car** 인터페이스만 사용하는 것을 확인할 수 있다.
- **Car**: 자동차의 역할이고 인터페이스이다. **K3Car**, **Model3Car** 클래스가 인터페이스를 구현한다.

```

package poly.car1;

public interface Car {
    void startEngine();
    void offEngine();
    void pressAccelerator();
}
  
```

```

package poly.car1;

public class K3Car implements Car {
    @Override
    public void startEngine() {
        System.out.println("K3Car.startEngine");
    }
}
  
```

```

@Override
public void offEngine() {
    System.out.println("K3Car.offEngine");
}

@Override
public void pressAccelerator() {
    System.out.println("K3Car.pressAccelerator");
}
}

```

```

package poly.car1;

public class Model3Car implements Car {
    @Override
    public void startEngine() {
        System.out.println("Model3Car.startEngine");
    }

    @Override
    public void offEngine() {
        System.out.println("Model3Car.offEngine");
    }

    @Override
    public void pressAccelerator() {
        System.out.println("Model3Car.pressAccelerator");
    }
}

```

```

package poly.car1;

public class Driver {

    private Car car;
}

```

```

public void setCar(Car car) {
    System.out.println("자동차를 설정합니다: " + car);
    this.car = car;
}

public void drive() {
    System.out.println("자동차를 운전합니다.");
    car.startEngine();
    car.pressAccelerator();
    car.offEngine();
}
}

```

- `Driver`는 멤버 변수로 `Car car`를 가진다.
- `setCar(Car car)`: 멤버 변수에 자동차를 설정한다. 외부에서 누군가 이 메서드를 호출해주어야 `Driver`는 새로운 자동차를 참조하거나 변경할 수 있다.
- `drive()`: `Car` 인터페이스가 제공하는 기능들을 통해 자동차를 운전한다.

```

package poly.car1;

/**
 * 다형성을 활용한 런타임 변경
 * 런타임: 애플리케이션 실행 도중에 변경 가능
 */
public class CarMain1 {
    public static void main(String[] args) {
        Driver driver = new Driver();

        //차량 선택(k3)
        Car k3Car = new K3Car();
        driver.setCar(k3Car);
        driver.drive();

        //차량 변경(k3 -> model3)
        Car model3Car = new Model3Car();
        driver.setCar(model3Car);
        driver.drive();
    }
}

```

실행 결과

자동차를 설정합니다: poly.car.K3Car@24d46ca6

자동차를 운전합니다.

K3Car.startEngine

K3Car.pressAccelerator

K3Car.offEngine

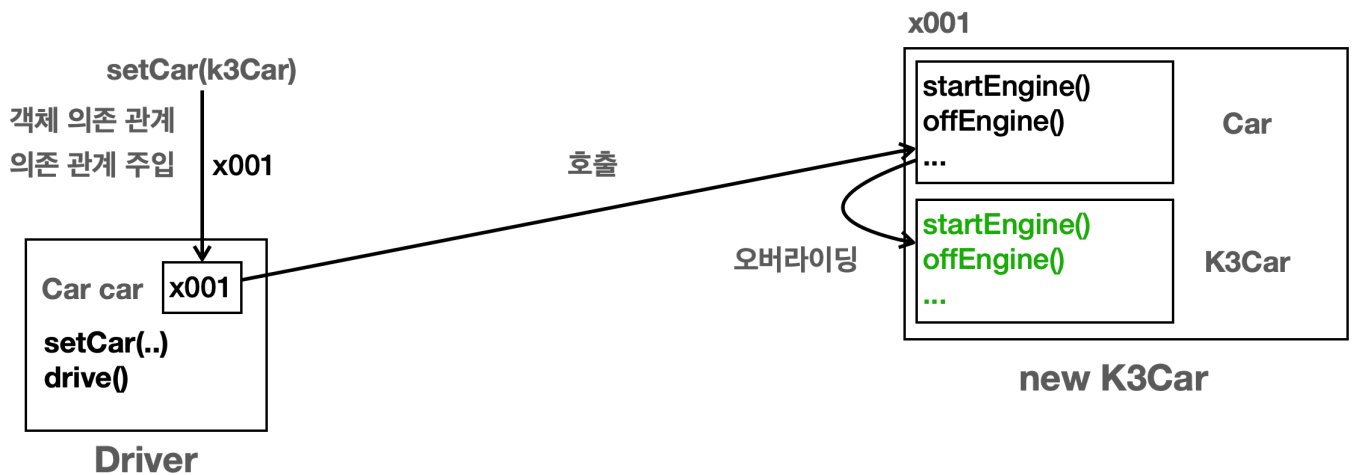
자동차를 설정합니다: poly.car.Model3Car@372f7a8d

자동차를 운전합니다.

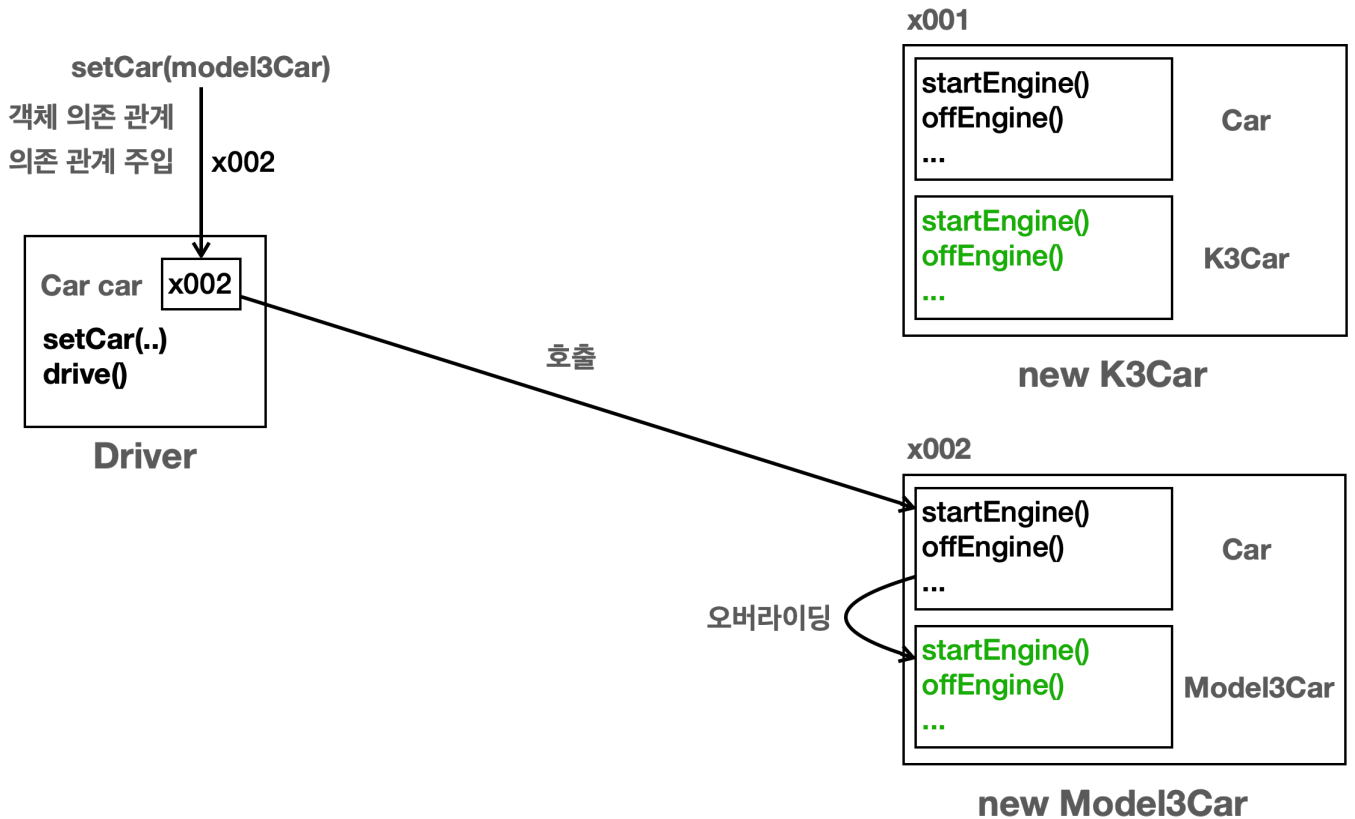
Model3Car.startEngine

Model3Car.pressAccelerator

Model3Car.offEngine



- 먼저 **Driver**와 **K3Car**를 생성한다.
- **driver.setCar(k3Car)**를 호출해서 **Driver**의 **Car car** 필드가 **K3Car**의 인스턴스를 참조하도록 한다.
- **driver.drive()**를 호출하면 **x001**을 참조한다. **car** 필드가 **Car** 타입이므로 **Car** 타입을 찾아서 실행하지만 메서드 오버라이딩에 의해 **K3Car**의 기능이 호출된다.



- **Model3Car** 를 생성한다.
- `driver.setCar(model3Car)` 를 호출해서 **Driver**의 **Car car** 필드가 **Model3Car**의 인스턴스를 참조하도록 변경한다.
- `driver.drive()` 를 호출하면 **x002**을 참조한다. **car** 필드가 **Car** 타입이므로 **Car** 타입을 찾아서 실행하지만 메서드 오버라이딩에 의해 **Model3Car**의 기능이 호출된다.

OCP(Open-Closed Principle) 원칙

좋은 객체 지향 설계 원칙 중 하나로 OCP 원칙이라는 것이 있다.

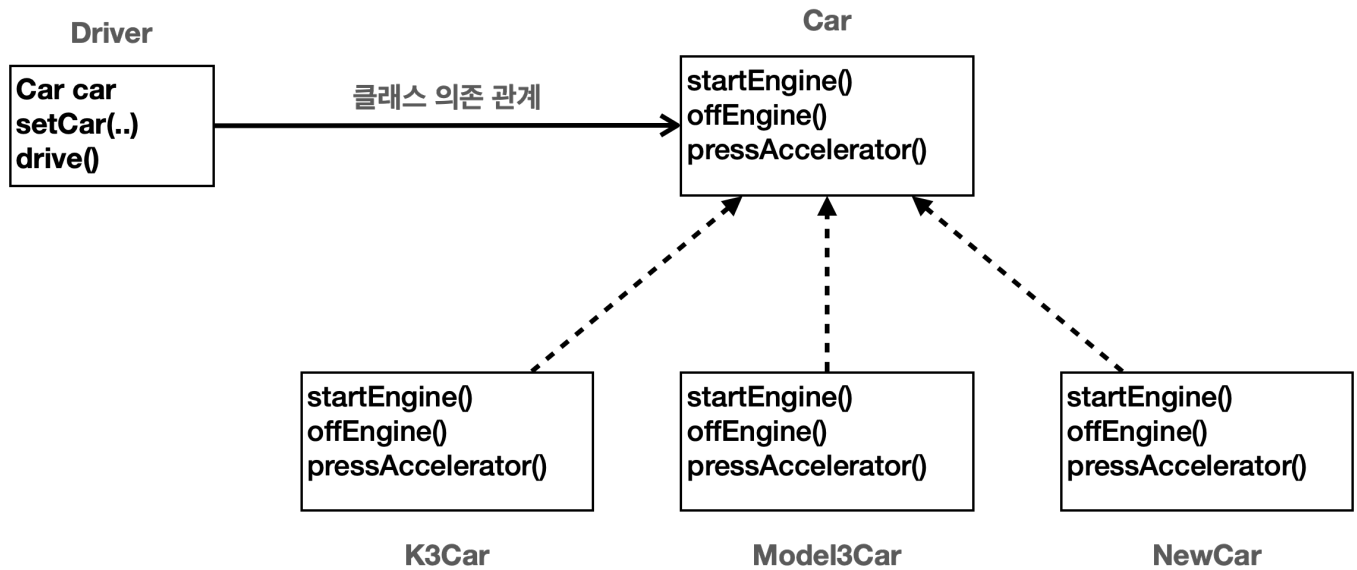
- **Open for extension:** 새로운 기능의 추가나 변경 사항이 생겼을 때, 기존 코드는 확장할 수 있어야 한다.
- **Closed for modification:** 기존의 코드는 수정되지 않아야 한다.

확장에는 열려있고, 변경에는 닫혀 있다는 뜻인데, 쉽게 이야기해서 기존의 코드 수정 없이 새로운 기능을 추가할 수 있다는 의미다. 약간 말이 안 맞는 것 같지만 우리가 앞서 개발한 코드가 바로 OCP 원칙을 잘 지키고 있는 코드다.

새로운 차량의 추가

여기서 새로운 차량을 추가해도 **Driver**의 코드는 전혀 변경하지 않는다. 운전할 수 있는 차량의 종류가 계속 늘어나도 **Car**를 사용하는 **Driver**의 코드는 전혀 변경하지 않는다. 기능을 확장해도 `main()` 일부를 제외한 프로그램의

핵심 부분의 코드는 전혀 수정하지 않아도 된다.



확장에 열려있다는 의미

Car 인터페이스를 사용해서 새로운 차량을 자유롭게 추가할 수 있다. Car 인터페이스를 구현해서 기능을 추가할 수 있다는 의미이다. 그리고 Car 인터페이스를 사용하는 클라이언트 코드인 Driver도 Car 인터페이스를 통해 새롭게 추가된 차량을 자유롭게 호출할 수 있다. 이것이 확장에 열려있다는 의미이다.

코드 수정은 닫혀 있다는 의미

새로운 차를 추가하게 되면 기능이 추가되기 때문에 기존 코드의 수정은 불가피하다. 당연히 어딘가의 코드는 수정해야 한다.

변하지 않는 부분

새로운 자동차를 추가할 때 가장 영향을 받는 중요한 클라이언트는 바로 Car의 기능을 사용하는 Driver이다. 핵심은 Car 인터페이스를 사용하는 클라이언트인 Driver의 코드를 수정하지 않아도 된다는 뜻이다.

변하는 부분

main()과 같이 새로운 차를 생성하고 Driver에게 필요한 차를 전달해주는 역할은 당연히 코드 수정이 발생한다. main()은 전체 프로그램을 설정하고 조율하는 역할을 한다. 이런 부분은 OCP를 지켜도 변경이 필요하다.

정리

- Car를 사용하는 클라이언트 코드인 Driver 코드의 변경없이 새로운 자동차를 확장할 수 있다.
- 다형성을 활용하고 역할과 구현을 잘 분리한 덕분에 새로운 자동차를 추가해도 대부분의 핵심 코드들을 그대로 유지할 수 있게 되었다.

전략 패턴(Strategy Pattern)

디자인 패턴 중에 가장 중요한 패턴을 하나 뽑으라고 하면 전략 패턴을 뽑을 수 있다. 전략 패턴은 알고리즘을 클라이언트 코드의 변경 없이 쉽게 교체할 수 있다. 방금 설명한 코드가 바로 전략 패턴을 사용한 코드이다. Car 인터페이스가 바로 전략을 정의하는 인터페이스가 되고, 각각의 차량이 전략의 구체적인 구현이 된다. 그리고 전략을 클라이언트 코드(Driver)의 변경 없이 손쉽게 교체할 수 있다.

문제와 풀이

문제1: 다중 메시지 발송

한번에 여러 곳에 메시지를 발송하는 프로그램을 개발하자.

다음 코드를 참고해서 클래스를 완성하자

요구사항

- 다형성을 활용하세요.
- Sender 인터페이스를 사용하세요.
- EmailSender, SmsSender, FaceBookSender 를 구현하세요.

```
package poly.ex.sender;

public class SendMain {

    public static void main(String[] args) {
        Sender[] senders = {new EmailSender(), new SmsSender(), new
        FaceBookSender()};
        for (Sender sender : senders) {
            sender.sendMessage("환영합니다!");
        }
    }
}
```

실행 결과

메일을 발송합니다: 환영합니다!

SMS를 발송합니다: 환영합니다!
페이스북에 발송합니다: 환영합니다!

정답

```
package poly.ex.sender;

public interface Sender {
    void sendMessage(String message);
}
```

```
package poly.ex.sender;

public class EmailSender implements Sender {
    @Override
    public void sendMessage(String message) {
        System.out.println("메일을 발송합니다: " + message);
    }
}
```

```
package poly.ex.sender;

public class SmsSender implements Sender {
    @Override
    public void sendMessage(String message) {
        System.out.println("SMS를 발송합니다: " + message);
    }
}
```

```
package poly.ex.sender;

public class FaceBookSender implements Sender {
    @Override
```

```
public void sendMessage(String message) {  
    System.out.println("페이스북에 발송합니다: " + message);  
}  
}
```

문제2: 결제 시스템 개발

여러분은 기대하던 결제 시스템 개발팀에 입사하게 되었다.

이 팀은 현재 2가지 결제 수단을 지원한다. 앞으로 5개의 결제 수단을 추가로 지원할 예정이다.

새로운 결제수단을 쉽게 추가할 수 있도록, 기존 코드를 리팩토링해라.

요구사항

- OCP 원칙을 지켜세요.
- 메서드를 포함한 모든 코드를 변경해도 됩니다. 클래스나 인터페이스를 추가해도 됩니다.
- 단 프로그램을 실행하는 PayMain0 코드는 변경하지 않고, 그대로 유지해야 합니다.
- 리팩토링 후에도 실행 결과는 기존과 같아야 합니다.

```
package poly.ex.pay0;  
  
public class KakaoPay {  
  
    public boolean pay(int amount) {  
        System.out.println("카카오페이 시스템과 연결합니다.");  
        System.out.println(amount + "원 결제를 시도합니다.");  
        return true;  
    }  
  
}
```

```
package poly.ex.pay0;  
  
public class NaverPay {  
  
    public boolean pay(int amount) {  
        System.out.println("네이버페이 시스템과 연결합니다.");  
        System.out.println(amount + "원 결제를 시도합니다.");  
    }  
  
}
```

```
        return true;
    }
}
```

```
package poly.ex.pay0;

public class PayService {

    public void processPay(String option, int amount) {

        boolean result;
        System.out.println("결제를 시작합니다: option=" + option + ", amount=" +
amount);
        if (option.equals("kakao")) {
            KakaoPay kakaoPay = new KakaoPay();
            result = kakaoPay.pay(amount);
        } else if (option.equals("naver")) {
            NaverPay naverPay = new NaverPay();
            result = naverPay.pay(amount);
        } else {
            System.out.println("결제 수단이 없습니다.");
            result = false;
        }

        if (result) {
            System.out.println("결제가 성공했습니다.");
        } else {
            System.out.println("결제가 실패했습니다.");
        }
    }

}
```

```
package poly.ex.pay0;

public class PayMain0 {

    public static void main(String[] args) {
        PayService payService = new PayService();
    }
}
```

```

        //kakao 결제
        String payOption1 = "kakao";
        int amount1 = 5000;
        payService.processPay(payOption1, amount1);

        //naver 결제
        String payOption2 = "naver";
        int amount2 = 10000;
        payService.processPay(payOption2, amount2);

        //잘못된 결제 수단 선택
        String payOption3 = "bad";
        int amount3 = 15000;
        payService.processPay(payOption3, amount3);
    }
}

```

실행 결과

결제를 시작합니다: option=kakao, amount=5000
 카카오페이 시스템과 연결합니다.
 5000원 결제를 시도합니다.
 결제가 성공했습니다.
 결제를 시작합니다: option=naver, amount=10000
 네이버페이 시스템과 연결합니다.
 10000원 결제를 시도합니다.
 결제가 성공했습니다.
 결제를 시작합니다: option=bad, amount=15000
 결제 수단이 없습니다.
 결제가 실패했습니다.

정답

이 문제에 정답은 없습니다.

새로운 결제 수단을 추가했을 때 `Pay`를 사용하는 클라이언트 코드인 `PayService`의 변경을 최소화 할 수 있다면 성공입니다.

```
package poly.ex.pay1;

public interface Pay {
    boolean pay(int amount);
}
```

```
package poly.ex.pay1;

public class KakaoPay implements Pay {
    @Override
    public boolean pay(int amount) {
        System.out.println("카카오페이 시스템과 연결합니다.");
        System.out.println(amount + "원 결제를 시도합니다.");
        return true;
    }
}
```

```
package poly.ex.pay1;

public class NaverPay implements Pay {
    @Override
    public boolean pay(int amount) {
        System.out.println("네이버페이 시스템과 연결합니다.");
        System.out.println(amount + "원 결제를 시도합니다.");
        return true;
    }
}
```

```
package poly.ex.pay1;

public class DefaultPay implements Pay {
    @Override
    public boolean pay(int amount) {
        System.out.println("결제 수단이 없습니다.");
        return false;
    }
}
```

```
}  
}
```

- 결제 수단을 찾지 못했을 때 사용한다. DefaultPay 사용하면 null 체크를 피할 수 있다.

```
package poly.ex.pay1;  
  
public abstract class PayStore {  
  
    //결제 수단 추가시 변하는 부분  
    public static Pay findPay(String option) {  
        if (option.equals("kakao")) {  
            return new KakaoPay();  
        } else if (option.equals("naver")) {  
            return new NaverPay();  
        } else {  
            return new DefaultPay();  
        }  
    }  
}
```

- 결제 수단 이름으로 실제 결제 수단 구현체를 찾는다.
- static으로 기능을 제공한다. 추상 클래스로 선언해서 객체 생성을 막는다.
- 결제 수단을 찾지 못했을 때 null 대신에 항상 실패하는 결제 수단을 사용한다.

```
package poly.ex.pay1;  
  
public class PayService {  
  
    public void processPay(String option, int amount) {  
        System.out.println("결제를 시작합니다: option=" + option + ", amount=" +  
amount);  
  
        Pay pay = PayStore.findPay(option);  
        boolean result = pay.pay(amount);  
  
        if (result) {  
            System.out.println("결제가 성공했습니다.");  
        } else {  

```

```

        System.out.println("결제가 실패했습니다.");
    }
}
}

```

- PayService 는 구체적인 결제 수단이 아니라 Pay 에 의존한다.
- 따라서 결제수단을 추가해도 PayService 의 코드에는 변경이 없다.

```

package poly.ex.pay1;

public class PayMain1 {

    public static void main(String[] args) {
        PayService payService = new PayService();

        //kakao 결제
        String payOption1 = "kakao";
        int amount1 = 5000;
        payService.processPay(payOption1, amount1);

        //naver 결제
        String payOption2 = "naver";
        int amount2 = 10000;
        payService.processPay(payOption2, amount2);

        //잘못된 결제 수단 선택
        String payOption3 = "bad";
        int amount3 = 15000;
        payService.processPay(payOption3, amount3);
    }
}

```

실행 결과

결제를 시작합니다: option=kakao, amount=5000
 카카오페이 시스템과 연결합니다.
 5000원 결제를 시도합니다.
 결제가 성공했습니다.

결제를 시작합니다: option=naver, amount=10000
네이버페이 시스템과 연결합니다.
10000원 결제를 시도합니다.
결제가 성공했습니다.
결제를 시작합니다: option=bad, amount=15000
결제 수단이 없습니다.
결제가 실패했습니다.

문제3: 결제 시스템 개발 - 사용자 입력

기존 결제 시스템이 사용자 입력을 받도록 수정하세요.

결제 수단을 입력하세요:kakao
결제 금액을 입력하세요:5000
결제를 시작합니다: option=kakao, amount=5000
카카오페이 시스템과 연결합니다.
5000원 결제를 시도합니다.
결제가 성공했습니다.
결제 수단을 입력하세요:exit
프로그램을 종료합니다.

정답

```
package poly.ex.pay1;

import java.util.Scanner;

public class PayMain2 {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        PayService payService = new PayService();

        while (true) {
            System.out.print("결제 수단을 입력하세요:");
            String payOption = scanner.nextLine();
            if (payOption.equals("exit")) {
                System.out.println("프로그램을 종료합니다.");
            }
        }
    }
}
```



```
        return;
    }
    System.out.print("결제 금액을 입력하세요:");
    int amount = scanner.nextInt();
    scanner.nextLine();

    payService.processPay(payOption, amount);
}
}
}
```

정리